Database Management Systems Project

Library Management System

Final Project Report

Team Members

L Srinath (24BDS0265)
Sugash K (24BCE2565)

Abstract

This project implements a library management application using MongoDB and Node.js. It manages books, authors, members, loans and reservations through a responsive staff interface. The database contains 137 seeded documents and uses validation, indexes, aggregation pipelines and transactions. Authentication and role checks protect staff operations. Automated tests verify fine calculations, authentication and atomic issue-return workflows.

Project objectives

- Maintain book, author and member information.
- Track issue, return, availability, overdue fines and reservation queues.
- Provide fast search and useful operational reports.
- Protect database consistency and staff-only operations.
- Provide a responsive interface for classroom demonstration.

1. Requirements

ID	Functional requirement
FR1	Add, update, search and control deletion of book records
FR2	Register, search and update library members
FR3	Issue an available book to an active member
FR4	Return a book and calculate overdue fine
FR5	Reserve a book when no copy is available
FR6	Maintain loan and reservation history
FR7	Display inventory and circulation analytics
FR8	Authenticate staff and enforce roles
FR9	Export overdue loan evidence as CSV

Non-functional requirements

- Responsive interface for desktop, tablet and mobile screens.
- Indexed search and bounded list queries.
- Strict validation and transaction consistency.
- Secure password hashing and HTTP-only session cookies.
- Repeatable Docker and local setup.

2. System architecture

Layer	Technology	Responsibility
Interface	HTML, CSS and JavaScript	Staff workflow and responsive rendering
API	Node.js and Express	Routes, validation, authorization and errors
Data models	Mongoose	Schemas, relationships and indexes
Database	MongoDB replica set	Documents, aggregations and transactions
Deployment	Docker Compose	Application and replica-set services
Verification	Node test, Supertest, memory replica set	Unit and integration tests

Request flow

Browser request - Express middleware - Authentication - Route handler - Mongoose model - MongoDB - JSON response - Updated interface.

3. Database design

Relationship	MongoDB representation
Author writes Book	Book stores an array of author IDs
Member borrows Book	Loan stores bookId and memberId
Member reserves Book	Reservation stores bookId and memberId
Staff issues Loan	Loan stores issuedBy user ID

Data volume

authors	books	members	loans	reservations	users	total
20	40	30	30	15	2	137

4. Core modules

Module	Capabilities
Authentication	Login, logout, JWT verification and admin/librarian roles
Catalog	Create, search, edit, stock display and controlled delete
Members	Register, search, edit, activate and controlled delete
Circulation	Issue, return, due dates, fines and history
Reservations	Availability rule, ordered queue and cancellation
Analytics	Six live aggregation reports and CSV export

5. NoSQL features

Indexes

The application uses unique, text, compound and partial indexes. They prevent duplicate ISBNs and duplicate active records while accelerating catalog, loan and reservation queries.

Aggregation

Six pipelines combine grouping, computed values, date differences and collection lookups to produce inventory, popularity, overdue, member, monthly and queue reports.

Transactions

Issue and return operations run on a MongoDB replica set. Every related update commits or rolls back as one unit.

Validation

Mongoose schemas validate application input. MongoDB JSON Schema validators apply strict rules to all six collections.

6. Security

- bcrypt password hashing with a work factor of 12 for seeded accounts.
- Signed JWT stored in an HTTP-only same-site cookie.
- Admin and librarian role checks on protected routes.
- Helmet security headers and request rate limiting.
- Environment variables for database URI and JWT secret.
- No secrets committed to the repository.

Production note

The seeded password exists only for classroom evaluation. A public deployment must change passwords, use TLS, store secrets in a secret manager and enable managed backups.

7. User interface

The interface is designed for library staff and uses a catalog-inspired visual system. A persistent desktop navigation becomes a mobile drawer. Tables can scroll horizontally on narrow screens, and forms use keyboard-visible focus styles.

Screen	Purpose
Login	Authenticate staff
Overview	Display counts, overdue status and inventory
Books	Search and manage catalog records
Members	Manage borrower records
Circulation	Issue and return books
Reservations	Manage the hold queue
Reports	Display all six aggregation results

8. Testing and results

Test	Result
JavaScript syntax check	Passed for application and browser code
Fine calculation unit tests	4 passed
Replica-set integration tests	2 passed
Issue and return transaction	Committed and verified
Seed data counts	137 records verified
Database validators	6 strict validators verified
Desktop browser check	Login, dashboard, books and reports passed
Mobile browser check	390-pixel viewport passed
Browser console	No errors detected

9. Deployment and operation

```
docker compose up -d --build
docker compose exec app npm run seed
open http://localhost:3000
```

The Docker configuration creates a MongoDB single-node replica set, initializes it, starts the application and exposes the interface on port 3000. The seed command can reset the classroom dataset at any time.

Demonstration credentials

Role	Email	Password
Administrator	admin@library.edu	admin123
Librarian	librarian@library.edu	admin123

10. Limitations and future work

- The project uses classroom seed data rather than a real institutional catalog.
- Email notifications and barcode scanning are not enabled.
- A production deployment should use managed backups and centralized audit logging.
- Future work can add member self-service, email reminders, barcode labels and recommendation models.

Conclusion

The project converts the Review 1 design into a complete, tested MongoDB application. It meets the minimum technical requirements and adds authentication, transactions, reports, responsive usability, export and repeatable deployment. The supplied documentation, user manual and presentation support the final demonstration and viva.